

Koopman-Based Control of a Soft Continuum Manipulator Under Variable Loading Conditions

Daniel Bruder[✉], Xun Fu, R. Brent Gillespie[✉], C. David Remy, and Ram Vasudevan[✉]

Abstract—Controlling soft continuum manipulator arms is difficult due to their infinite degrees of freedom, nonlinear material properties, and large deflections under loading. This letter presents a data-driven approach to identifying soft manipulator models that enables consistent control under variable loading conditions. This is accomplished by incorporating loads into a linear Koopman operator model as states and estimating their values online via an observer within the control loop. Using this approach, autonomous control of a pneumatically actuated soft continuum manipulator is achieved. In several trajectory following experiments, this controller is shown to be more accurate and precise than controllers based on models that do not explicitly account for loading. The manipulator also successfully performs pick and place of objects with unknown mass, demonstrating the efficacy of this approach in executing real-world manipulation tasks.

Index Terms—Soft robotics, Manipulators, Model learning for control.

I. INTRODUCTION

SOFT continuum manipulator arms are lightweight, inexpensive to fabricate, and their inherent compliance carries the promise to enable safe interaction with humans, delicate objects, and the environment [10], [15], [16], [22], [24]. These properties would make them ideal platforms for tasks involving physical human-robot interaction such as feeding [23] or handling products in a warehouse [12]. Yet, so far, the real-world application of soft manipulators has been limited. This is due to the difficulty involved in controlling such systems, as they exhibit infinite degrees-of-freedom, nonlinear material properties, and large deflections under loading [13]. These characteristics greatly complicate manipulation tasks such as pick and place, which require consistent control performance regardless of payload. Control-oriented models that describe soft manipulator behavior

under varying loading conditions could enable the automation of such tasks.

Recently, data-driven modeling techniques have emerged as a powerful tool to address the challenge of modeling soft continuum manipulators. Within the class of data-driven methods, deep learning of neural networks have been the primary choice for describing the input/output behavior of soft manipulators. For instance, Satheeshbabu *et al.* [26] used deep reinforcement learning to achieve open-loop position control of a soft manipulator comprised of fiber-reinforced actuators; Hyatt *et al.* [17] utilized a linearization of a neural network model and model predictive control to control the position of a bellows-actuated manipulator; and Thuruthel *et al.* [27] used a combination of a recurrent neural network and supervised reinforcement learning to achieve closed-loop control of a pneumatically-driven soft manipulator. This controller was shown to compensate for disturbance such as end effector loading. Moving forward, models that are able to predict system behavior as a function of load rather than treating them as a disturbance are required to improve system performance with regard to accuracy and speed.

An alternative to deep learning and artificial neural networks is a data-driven modeling approach based on Koopman operator theory. This approach yields a linear model that can be used for control. These properties are in stark contrast to deep learning or neural network models, whose creation requires solving a nonlinear optimization problem for which global convergence may not be guaranteed [4], and in which the control input usually appears non-linearly. The Koopman operator, on which this approach is based, describes the evolution of scalar-valued functions along trajectories of a nonlinear dynamical system, acting as a linear embedding of the nonlinear dynamics in an infinite-dimensional function space [9], [8, Ch. 7]. An approximation of this infinite-dimensional operator is identified via linear regression on input/output data [7], [21], providing a linear model representation that is compatible with established linear control techniques [2], [18]. A Koopman-based approach has been successfully used to control several robotic systems such as a Sphero SPRK, [2], a quadcopter [1], and a robotic fish [19]. Koopman-based system identification and control has also been successfully demonstrated on a soft continuum manipulator to control the 2D projection of its end-effector [6].

This letter presents a Koopman-based framework that explicitly incorporates loading conditions into the model to enable real-time control design. By incorporating loads into the model as states, our approach is able to estimate loading online via an observer within the control loop (see Fig. 1). This observer infers

Manuscript received February 24, 2021; accepted June 16, 2021. Date of publication July 7, 2021; date of current version July 23, 2021. This letter was recommended for publication by Associate Editor H. Hauser and Editor C. Laschi upon evaluation of the reviewers' comments. This work was supported in part by the Toyota Research Institute, and in part by the National Science Foundation Graduate Research Fellowship Program under Grant 1256260 DGE. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation. (*Corresponding author: Daniel Bruder.*)

Daniel Bruder is with the School of Engineering and Applied Sciences, Harvard University, Cambridge, MA 02138, USA (e-mail: dbruder@seas.harvard.edu).

Xun Fu, R. Brent Gillespie, and Ram Vasudevan are with the Department of Mechanical Engineering, University of Michigan, Ann Arbor, MI 48109, USA (e-mail: xunfu@umich.edu; brentg@umich.edu; ramv@umich.edu).

C. David Remy is with the Institute for Nonlinear Mechanics, University of Stuttgart, 70569 Stuttgart, Germany (e-mail: remy@inm.uni-stuttgart.de).

Digital Object Identifier 10.1109/LRA.2021.3095268

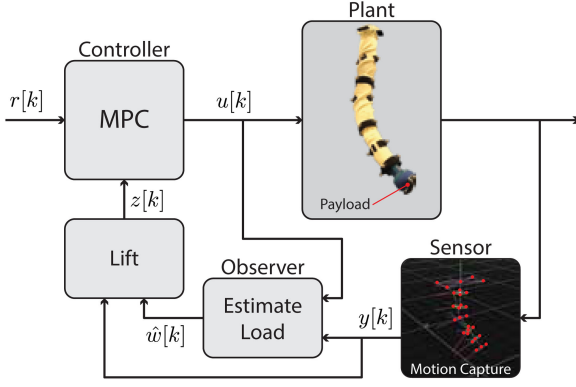


Fig. 1. A soft continuum manipulator is tasked with following a reference trajectory $r[k]$ while carrying an unknown payload. At each time step, a model predictive controller computes the optimal input $u[k]$ to follow the reference trajectory based on a linear Koopman operator model and the position of the end effector $y[k]$, which is measured by a motion capture system. An observer computes a payload estimate $\hat{w}[k]$ based on the previously measured inputs and outputs, which is then incorporated into the state of the model $z[k]$ via a lifting function.

the most likely value of the loading condition given a series of input/output measurements. The knowledge that is gained in this process enables consistent control performance under a wide range of loading conditions. In fact, the idea of estimating loading conditions has been explored for rigid-bodied robots in the past [11], [31]. By using our proposed Koopman-based approach, we are able to transfer these rigid-body results into the world of soft robots, which are subject to continuous deformation under load.

Using this approach, we demonstrate real-time, fully autonomous control of a pneumatically actuated soft continuum manipulator. In several validation experiments our controller proves itself to be more accurate and more precise across various payloads than several other model-based controllers which do not incorporate loading. To the best of the authors' knowledge, this letter presents the first implementation of a closed-loop controller that explicitly accounts for loading on a soft continuum manipulator and the first demonstration of autonomous pick and place of objects of unknown mass on a soft continuum manipulator.

II. SYSTEM IDENTIFICATION

This section describes a system identification method to construct linear state space models of nonlinear controlled discrete dynamical systems from input/output data using the Koopman operator.

A. The Koopman Operator

Consider an input/output system governed by the following differential equation for the output:

$$\dot{y}(t) = F(y(t), u(t)) \quad (1)$$

where $y(t) \in Y \subset \mathbb{R}^n$ is the output and $u(t) \in U \subset \mathbb{R}^m$ is the input of the system at time $t \geq 0$, F is a continuously differentiable function, and Y and U are compact subsets. Denote by $\phi_\tau(y_0, u_0)$ the *flow map*, which is the solution to (1) at time

τ when beginning with the initial condition y_0 at time 0 and a constant input u_0 applied for all time between 0 and τ .

The system can be lifted to an infinite-dimensional function space $\mathcal{F}$ composed of all square-integrable real-valued functions with compact domain $Y \times U \subset \mathbb{R}^n \times \mathbb{R}^m$. Elements of $\mathcal{F}$ are called *observables*. In $\mathcal{F}$, the flow of the system is characterized by the set of Koopman operators $\mathcal{K}_\tau : \mathcal{F} \rightarrow \mathcal{F}$, for each $\tau \geq 0$, which describe the evolution of the observables $f \in \mathcal{F}$ along the trajectories of the system according to the following definition:

$$\mathcal{K}_\tau f = f \circ \phi_\tau, \quad (2)$$

where $\circ$ indicates function composition. A consequence of this definition is that for a specific time step τ , the Koopman operator $\mathcal{K}_\tau$ defines an infinite-dimensional linear discrete dynamical system that advances the value of an observable by τ ,

$$f(y(t + \tau), \tilde{u}) = \mathcal{K}_\tau f(y(t), \tilde{u}) \quad (3)$$

where $\tilde{u}$ is a constant input over the interval $[t, t + \tau]$. Since this is true for *any* observable function f , the Koopman operator can be used to advance the output itself by applying it to the set of functions $\{f_i : f_i(y(t), \tilde{u}) = y_i(t)\}_{i=1}^n$, advancing their values according to (3), and stacking the results as a vector:

$$y(t + \tau) = \begin{bmatrix} \mathcal{K}_\tau f_1(y(t), \tilde{u}) & \cdots & \mathcal{K}_\tau f_n(y(t), \tilde{u}) \end{bmatrix}^\top. \quad (4)$$

In this way, the Koopman operator provides an infinite-dimensional linear representation of a nonlinear dynamical system [9].

B. Koopman-Based System Identification

Since the Koopman operator is an infinite-dimensional object, we have to settle for its projection onto a finite-dimensional subspace, which can be represented as a matrix. Using the Extended Dynamic Mode Decomposition (EDMD) algorithm [21], [30], we identify a finite-dimensional matrix approximation of the Koopman operator via linear regression applied to observed data. The remainder of this subsection describes this process.

Define $\bar{\mathcal{F}} \subset \mathcal{F}$ to be the subspace of $\mathcal{F}$ spanned by $N > n + m$ linearly independent basis functions $\{\psi_i : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}\}_{i=1}^N$, and define the *lifting function* $\psi : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^N$ as:

$$\psi(y, u) := \begin{bmatrix} \psi_1(y, u) & \cdots & \psi_N(y, u) \end{bmatrix}^\top. \quad (5)$$

To approximate the Koopman operator from a set of experimental data, we take K discrete measurements in the form of so-called “snapshots” $(a[k], b[k], u[k])$ for each $k \in \{1, \dots, K\}$ where

$$a[k] := y[k] \quad (6)$$

$$b[k] := \phi_{T_s}(y[k], u[k]), \quad (7)$$

$y[k]$ denotes the output corresponding to the k^{th} measurement, $u[k]$ is the constant input applied between $a[k]$ and $b[k]$, and T_s is the sampling period, which is assumed to be identical for all snapshots. Note that consecutive snapshots do not have to be generated by consecutive measurements. We then lift all of the snapshots according to (5) and compile them into the following

$K \times N$ matrices:

$$\Psi_a := \begin{bmatrix} \psi(a[1], u[1])^\top \\ \vdots \\ \psi(a[K], u[K])^\top \end{bmatrix}, \quad \Psi_b := \begin{bmatrix} \psi(b[1], u[1])^\top \\ \vdots \\ \psi(b[K], u[K])^\top \end{bmatrix}. \quad (8)$$

$\bar{\mathcal{K}}_{T_s}$ is chosen so that it yields the least-squares best fit to all of the observed data, which is given by

$$\bar{\mathcal{K}}_{T_s} := \Psi_a^\dagger \Psi_b. \quad (9)$$

where superscript $\dagger$ denotes the Moore-Penrose pseudoinverse.

Sometimes a more accurate model can be obtained by incorporating delays into the set of snapshots. To incorporate these delays, we modify the snapshots to have the following form

$$a[k] := \begin{bmatrix} y[k] \\ \vdots \\ y[k-d] \\ u[k-1] \\ \vdots \\ u[k-d] \end{bmatrix}, \quad b[k] := \begin{bmatrix} \phi_{T_s}(y[k], u[k]) \\ \vdots \\ y[k-d+1] \\ u[k-1] \\ \vdots \\ u[k-d] \end{bmatrix} \quad (10)$$

where d is the number of delays. We then modify the domain of the lifting function such that $\psi : \mathbb{R}^{(n+(n+m)d)} \times \mathbb{R}^m \rightarrow \mathbb{R}^N$ to accommodate the larger dimension of the snapshots. Once these snapshots have been assembled, the model identification procedure is identical to the case without delays.

C. Linear Model Realization Based on the Koopman Operator

For dynamical systems with inputs, we are interested in using the Koopman operator to construct discrete linear models of the following form

$$\begin{aligned} z[j+1] &= Az[j] + Bu[j] \\ y[j] &= Cz[j] \end{aligned} \quad (11)$$

for each $j \in \mathbb{N}$, where $y[0]$ is the initial output, $z[0]$ is the initial state, and $u[j] \in \mathbb{R}^m$ is the input at the j^{th} step. Specifically, we desire a representation in which the input appears *linearly*, because models of this form are amenable to real-time, convex optimization techniques for feedback control design, as we describe in Section III.

With a suitable choice of basis functions $\{\psi_i\}_{i=1}^N$, $\bar{\mathcal{K}}_{T_s}$ can be constructed such that it is decomposable into a linear system representation like (11). One way to achieve this is to define the first $N-m$ basis functions as functions of the output only, and the last m basis functions as indicator functions on each component of the input,

$$\psi_i(y, u) = g_i(y), \quad \forall i \in \{1, \dots, N-m\} \quad (12)$$

$$\psi_i(y, u) = u_i, \quad \forall i \in \{i = N-m+1, \dots, N\} \quad (13)$$

where $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ and u_i denotes the i^{th} element of u . This choice ensures that the input only appears in the last m components of $\psi(y, u)$, and an $N-m$ dimensional state can

be defined as $z = g(y) \in \mathbb{R}^{N-m}$, where $g : \mathbb{R}^n \rightarrow \mathbb{R}^{N-m}$ is defined as

$$g(y) := \begin{bmatrix} g_1(y) & \cdots & g_{N-m}(y) \end{bmatrix}^\top. \quad (14)$$

Following from (9), the transpose of $\bar{\mathcal{K}}_{T_s}$ is the best transition matrix between the elements of the lifted snapshots in the L^2 -norm sense. This implies that given the lifting functions defined in (12) and (13), $\bar{\mathcal{K}}_{T_s}$ is the minimizer to

$$\min_{\bar{\mathcal{K}}} \sum_{k=1}^K \left\| \bar{\mathcal{K}}^\top \begin{bmatrix} g(a[k]) \\ u[k] \end{bmatrix} - \begin{bmatrix} g(b[k]) \\ u[k] \end{bmatrix} \right\|_2^2. \quad (15)$$

Also note that given $z = g(y)$ as the state of our linear system, the best realizations of A and B in the L^2 -norm sense are the minimizers to

$$\min_{\bar{A}, \bar{B}} \sum_{k=1}^K \left\| \bar{A}g(a[k]) + \bar{B}u[k] - g(b[k]) \right\|_2^2. \quad (16)$$

Therefore, by comparing (15) and (16), one can confirm that A and B are embedded in $\bar{\mathcal{K}}_{T_s}$ and can be isolated by partitioning it as follows:

$$\bar{\mathcal{K}}_{T_s}^\top = \begin{bmatrix} A_{(N-m) \times (N-m)} & B_{(N-m) \times m} \\ \vdots & \vdots \end{bmatrix} \quad (17)$$

where the subscripts denote the dimensions of each matrix.

We can also define the first n basis functions as indicator functions on each component of the output, i.e.

$$g_i(y) = y_i \quad \forall i \in \{1, \dots, n\}. \quad (18)$$

Then, C is defined as the matrix which projects the first n elements of the state onto the output-space,

$$C = \begin{bmatrix} I_{n \times n} & O_{n \times (N-n)} \end{bmatrix}. \quad (19)$$

D. Incorporating Loading Conditions Into the Model

For robots that interact with objects or their environment, understanding the effect of external loading on their dynamics is critical for control. These loading conditions alter the dynamics of a system, but are generally not directly observable. This poses a challenge for model-based control, which relies on an accurate dynamical model to choose suitable control inputs for a given task.

We desire a way to incorporate loading conditions into our dynamic system model and to estimate them online. We can achieve this by including them within the states of our Koopman-based lifted system model, and then constructing an online observer to estimate their values. This strategy utilizes the underlying model to infer the most likely value of the loading conditions given past input/output measurements.

Let $w \in \mathbb{R}^p$ be a parametrization of loading conditions, as defined by the user. For example, w might specify the mass at the end effector of a manipulator arm, the magnitude and direction of external forces, or the direction of gravity [5, Section 5.2]. We incorporate w into the state z using a new lifting function $\gamma : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^{(N-m)(p+1)}$, which accepts w as a second

input and is defined as:

$$z = \gamma(y, w) = \begin{bmatrix} g(y) \\ g(y)w_1 \\ \vdots \\ g(y)w_p \end{bmatrix} = \underbrace{\begin{bmatrix} g(y) & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & g(y) \end{bmatrix}}_{\Gamma(y)} \begin{bmatrix} 1 \\ w \end{bmatrix} \quad (20)$$

where $\Gamma(y) \in \mathbb{R}^{(N-m)(p+1)} \times \mathbb{R}^{p+1}$ is the matrix formed by diagonally concatenating $g(y)$, $p+1$ times, and w_i denotes the i^{th} element of w . Note that because this lifting function requires the loading condition w as an input, it must also be included in the snapshots $(a[k], b[k], u[k], w[k])$ to construct a Koopman model that accounts for loading.

In the data used for model construction we assume that w is known. However, in control applications w will often be unknown or impractical to measure directly. In that case, its value can be inferred based on the identified system model and past input/output measurements. We construct an observer that estimates the value of w at the j^{th} timestep by solving a linear least-squares problem using data from the N_w previous timesteps. Notice that the output at the j^{th} timestep $y[j]$ can be expressed in terms of the input $u[j-1]$, the output $y[j-1]$, and the load $w[j-1]$ at the previous timestep by combining the system model equations of (11) and then substituting (20) for $z[j-1]$,

$$y[j] = CAz[j-1] + CBu[j-1] \quad (21)$$

$$= CA\Gamma(y[j-1]) \begin{bmatrix} 1 \\ w[j-1] \end{bmatrix} + CBu[j-1]. \quad (22)$$

Solving for the best estimate of $w[j-1]$ in the L^2 -norm sense, denoted $\hat{w}[j-1]$ yields the following expression,

$$\begin{bmatrix} 1 \\ \hat{w}[j-1] \end{bmatrix} = (CA\Gamma(y[j-1]))^\dagger (y[j] - CBu[j-1]). \quad (23)$$

Under the assumption that the loading is equal to some constant $\bar{w}$ over the previous N_w time steps, i.e. $w[i] = \bar{w}$ for $i = j - N_w, \dots, j$, we can similarly find the best estimate over all N_w timesteps. Since this estimate is based on more data, it should be more accurate and more robust to noisy output measurements. We define the following two matrices,

$$\Lambda_A = \begin{bmatrix} CA\Gamma(y[j-1]) \\ \vdots \\ CA\Gamma(y[j-N_w]) \end{bmatrix} \quad (24)$$

$$\Lambda_B = \begin{bmatrix} y[j] - CBu[j-1] \\ \vdots \\ y[j-N_w+1] - CBu[j-N_w] \end{bmatrix} \quad (25)$$

Then, following from (23), the best estimate for $\bar{w}$ over the past N_w timesteps in the L^2 -norm sense, denoted $\hat{w}$, is given by

$$\begin{bmatrix} 1 \\ \hat{w} \end{bmatrix} = \Lambda_A^\dagger \Lambda_B. \quad (26)$$

Algorithm 1: Koopman MPC with Load Estimation.

Input: Prediction horizon: N_h
Model matrices: A, B, C
Load estimation parameters: N_w, N_e, N_r

for $k = 0, 1, 2, \dots$ **do**
 if $k \bmod N_e = 0$ **then**
 Estimate $\hat{w}'[j]$ via (26)
 $\hat{w}[k] = \text{mean}(\hat{w}'[j], \hat{w}[k-1], \dots, \hat{w}[k-N_r])$
 $j = j + 1$
 else
 $\hat{w}[k] = \hat{w}[k-1]$
 end
 Step 1: Solve QP to find optimal input $(u[i]^*)_{i=0}^{N_h}$
 Step 2: Set $u[k] = u[0]^*$
 Step 3: Apply $u[k]$ to the system
end

III. CONTROL

A popular model-based control design technique is model predictive control (MPC), wherein one optimizes the control input over a finite time horizon, applies that input for a single timestep, and then optimizes again, repeatedly [25]. For linear systems, MPC consists of iteratively solving a convex quadratic program (QP). This is also the case for Koopman-based linear MPC control, wherein one solves for the optimal sequence of control inputs over a receding prediction horizon [18, Eq. 23].

The predictions of this Koopman-based controller depend on the estimate of the loading conditions $\hat{w}$. This estimate must be periodically updated using the method described in Section II-D, but for systems with relatively stable loading conditions, it is computationally inefficient to compute a new estimate at every time step. Therefore, we define a load estimation update period N_e as the number of time steps to wait between load estimations. To balance accuracy with responsiveness, we update $\hat{w}$ every N_e time steps by setting it equal to the average of the new load estimate and the previous N_r load estimates, where N_r is another user defined constant. Algorithm 1 summarizes the closed-loop operation of this Koopman-based MPC controller with these periodic load estimation updates.

IV. EXPERIMENTS

This section describes the soft continuum manipulator and the set of experiments used to demonstrate the efficacy of the modeling and control methods described in Sections II and III. Footage from these experiments is included in a supplementary video file¹.

A. System Identification of Soft Robot Arm

To validate the modeling and control approach described in the previous section, we applied it to a soft robot arm capable of picking-up objects and moving its end effector in three-dimensional space. The robot, shown in Fig. 2, is 70 cm long and has a diameter of 6 cm. It is made up of three pneumatically actuated bending sections and an end effector comprised of a granular jamming vacuum gripper [3]. Each section is actuated by three pneumatic artificial muscles (PAMs) [28] which are

¹<https://youtu.be/KZo8-bKJksY>

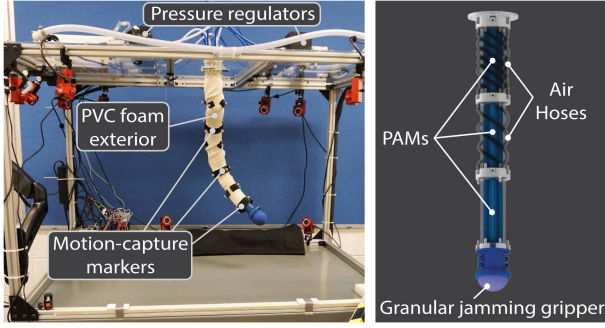


Fig. 2. The soft robot arm consists of three bending sections, each actuated by three pneumatic artificial muscles (PAMs). The actuators are surrounded by a sleeve of flexible PVC foam, and pressurized air is supplied to the actuators via air hoses that wind around the exterior. The end effector consists of a granular jamming vacuum gripper [3], which is connected to a vacuum pump by a hose that runs along the interior of the arm.

adhered to a central spine consisting of an air hose encased in flexible PVC foam tubing. Another much larger sleeve of flexible PVC foam surrounds the actuators, which serves to dampen high frequency oscillations and make the body of the arm softer overall. The air pressure inside the actuators is regulated by 9 Enfield TR-010-g 10-s pneumatic pressure regulators that accept 0–10V command signals corresponding to pressures of approximately 0–275 kPa, and are connected to the actuators by air hoses that wrap around the outside of the foam sleeves. The exterior of the arm is covered in retro-reflective markers which are tracked using a commercial OptiTrack motion capture system.

We quantified the stochastic behavior of our soft robot system by observing the variations in output from period-to-period under sinusoidal inputs with a period of 10 seconds and a sampling time of $T_s = 0.083$ seconds with a zero-order-hold between samples. Over 60 periods, the trajectory of the end effector deviated from the mean trajectory by an average of 9.45 mm and with a standard deviation of 7.3 mm. This inherent stochasticity limits the tracking performance of the system, independent of the employed controller.

For the purposes of constructing a dynamic model for the arm, the input was chosen to be the command voltages into the 9 pressure regulators and at each instance in time was restricted to $[0, 10]^9$. The output was chosen to live in $\mathbb{R}^9$ and corresponds to the positions of the ends of each of the 3 bending sections in Cartesian coordinates with the last 3 coordinates corresponding to the end effector position. The parametrization of the loading condition lives in $\mathbb{R}_+$ and is chosen to be the mass of the object held by the gripper.

Data for constructing models was collected over 49 trials lasting approximately 10 minutes each. A randomized “ramp and hold” type input and a load from the set $\{0, 50100150, 200250, 300\}$ grams was applied during each trial to generate a representative sampling of the system’s behavior over its entire operating range. Three models were fit from the data: a linear state-space model using the subspace method [29], a linear Koopman model that *does not* take loading into account using the approach described in Section II-B, and a linear Koopman model that *does* incorporate loading using the approach from Section II-D. Each of these models was fit using the

same set of 325 733 randomly generated data points just once, independent of any specific task.

The linear state-space model provides a baseline for comparison and was identified using the MATLAB System Identification Toolbox [20]. This model is a 9 dimensional linear state-space model expressed in observer canonical form.

The first Koopman model (without loading) was identified on a set of $K = 325732$ snapshot pairs $\{a[k], b[k], u[k]\}_{k=1}^K$ that incorporate a single delay $d = 1$:

$$a[k] = \begin{bmatrix} y[k]^\top & y[k-1]^\top & u[k-1]^\top \end{bmatrix}^\top \quad (27)$$

$$b[k] = \begin{bmatrix} \phi_{T_s}(y[k], u[k])^\top & y[k]^\top & u[k]^\top \end{bmatrix}^\top. \quad (28)$$

Note that the dimension of each snapshot is $2n + m = 2(9) + 9 = 27$ due to the inclusion of the delay, and we denote by $y^d[k] \in \mathbb{R}^{27}$ one of these outputs which has delays included at some time k . The lifting function $\psi : \mathbb{R}^{27} \times \mathbb{R}^9 \rightarrow \mathbb{R}^{111}$ was defined as

$$\psi(y^d[k], u[k]) = \begin{bmatrix} g(y^d[k]) \\ u[k] \end{bmatrix} \quad (29)$$

where the range of g has dimension 102, $g_i(y^d[k]) = y_i^d[k]$ for $i = 1, \dots, 27$, and the remaining 75 basis functions $\{g_i : \mathbb{R}^{27} \rightarrow \mathbb{R}\}_{i=28}^{102}$ are polynomials of maximum degree 2 that were selected by evaluating the snapshot pairs on the set of all monomials of degree less than or equal to 2, then performing principle component analysis (PCA) [8, Ch. 1.5] to identify a reduced set of polynomials that can still explain at least 99% of this lifted data.

The second Koopman model (with loading) was identified on the same set of snapshot pairs as the first model, but with the loading included $\{a[k], b[k], u[k], w[k]\}_{k=1}^K$. The lifting function $\psi : \mathbb{R}^{27} \times \mathbb{R}^9 \rightarrow \mathbb{R}^{231}$ was defined as,

$$\psi(y^d[k], u[k], w[k]) = \begin{bmatrix} \gamma(y^d[k], w[k]) \\ u[k] \end{bmatrix} = \begin{bmatrix} g(y^d[k]) \\ g(y^d[k])w[k] \\ u[k] \end{bmatrix} \quad (30)$$

where the range of g has dimension 111, $g_i(y^d[k]) = y_i^d[k]$ for $i = 1, \dots, 27$, and the remaining 84 basis functions $\{g_i : \mathbb{R}^n \rightarrow \mathbb{R}\}_{i=28}^{111}$ are polynomials of maximum degree 2 that were selected using the same PCA method described in the previous paragraph.

B. Description of Controllers

Three model predictive controllers were constructed using the data-driven models described in the previous section. Each controller uses one of the identified models to compute online predictions and is denoted by an abbreviation specifying which model,

- L-MPC: Uses the linear state-space model
- K-MPC: Uses the Koopman model without loading
- KL-MPC: Uses the Koopman model with loading

All three controllers solve a quadratic program at each time step using the Gurobi Optimization software [14]. They run in closed-loop at 12 Hz, feature an MPC horizon of 1 s ($N_h = 12$),

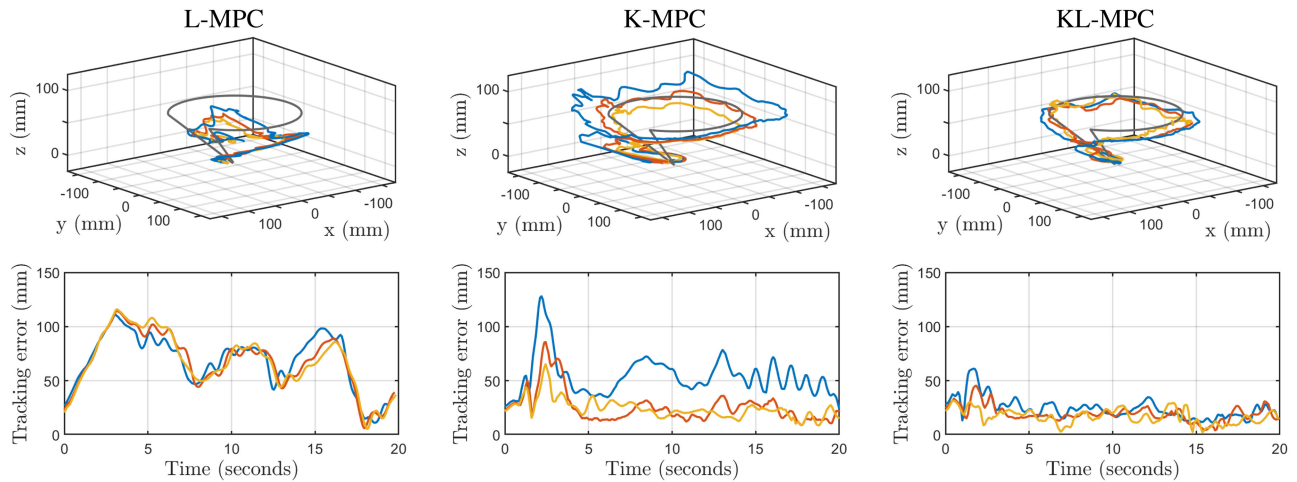


Fig. 3. Experiment 1 results: The end effector trajectories for the L-MPC (left), K-MPC (center), and KL-MPC (right) controllers when the true value of the payload is known. Trajectories corresponding to a payload of 25 g are shown in blue, trajectories with a payload of 125 g are shown in red, trajectories with a payload of 225 g are shown in yellow, and the reference trajectory is shown in grey.

TABLE I
EXPERIMENT 1: RMSE (MM) OVER ENTIRE TRIAL

Controller	Payloads (grams)						Avg.	Std. Dev.
	25	75	125	175	225	275		
L-MPC	73.0	72.9	72.6	71.9	72.3	74.3	72.8	0.8
K-MPC	55.4	33.9	29.5	20.0	24.8	27.8	31.9	12.4
KL-MPC	26.1	23.7	20.6	19.5	18.2	20.4	21.4	2.9

and a cost function that penalizes deviations of the position of the end effector from a reference trajectory over the prediction horizon.

C. Experiment 1: Trajectory Following With Known Payload

We first evaluated the relative performance of the three controllers when the payload at the end effector is known. With this information given, the manipulator is tasked with moving the end effector along a three-dimensional reference trajectory lasting 20 seconds. Six trials were completed for payloads of 25, 75, 125, 175, 225, and 275 grams. The actual paths traced out by the end effector and the tracking error over time for 3 of the trials are displayed in Fig. 3, and the RMSE tracking error for all 6 trials is compiled in Table I. It should be noted that only the KL-MPC controller is capable of actually utilizing knowledge of the payload, since the other 2 controllers are based on models that do not incorporate loading conditions.

D. Experiment 2: Trajectory Following With Unknown Payload

To evaluate the efficacy of the combined control and load estimation method summarized by Algorithm 1, we measured the manipulator's performance in tracking a periodic reference trajectory when the payload is not known. The periodic reference trajectory was a circle with a diameter of 200 mm and three trials were conducted with payloads of 25, 125, and 225 grams. Note that these payloads and trajectory were not included in the training data. The KL-MPC controller was run at 12 Hz,

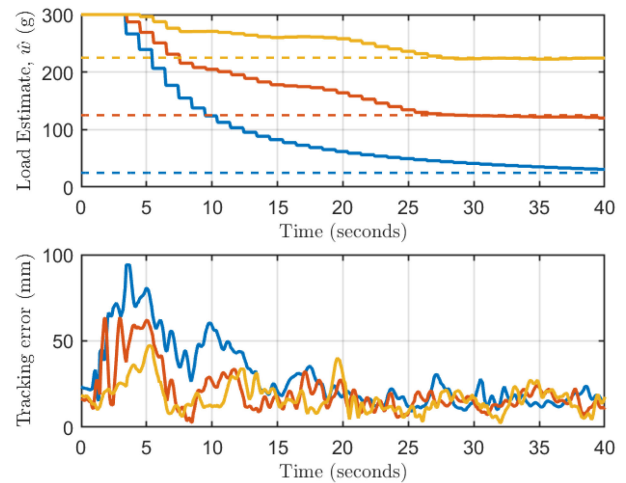


Fig. 4. Experiment 2 results: Periodic trajectory following with an unknown payload. The payload estimate over time (top) and tracking error over time (bottom) are shown for three trials with payloads of 25 g, 125 g, and 225 g. Results for the 25 g payload trial are shown in blue, results for the 125 g payload trial are shown in red, and results for the 225 g payload trial are shown in yellow. After 20 seconds, the error stays below 30 mm for the remainder of all three trials.

load estimates were calculated every $N_e = 12$ time steps by solving (26) using measurements from the previous $N_w = 30$ time steps, and $\hat{w}$ was computed by averaging over the most recent $N_r = 360$ estimates. Results of this experiment are shown in Fig. 4.

E. Experiment 3: Automated Object Sorting (Pick and Place)

The load estimation algorithm and KL-MPC controller were utilized to perform automated object sorting by mass. Five objects were selected, each with mass between 0 and 250 grams, and five cups were placed in front of the manipulator, each corresponding to a 50 g interval between 0 and 250 grams (i.e. 0-50, 50-100, etc.). The range from 250-300 grams was not used

	0-50 g	50-100 g	100-150 g	150-200 g	200-250 g
Trial 1	 23 g	 91 g	 133 g	 189 g	 229 g
Trial 2	 30 g	 97 g	 135 g	 166 g	 236 g

Fig. 5. Objects used for Experiment 3: In each trial, the soft manipulator sorted a set of five objects according to their mass, based on an estimate computed by the online observer described in Section II-D. The set of objects used for each trial are separated by row, and the mass of each object is written below it.

for this experiment, because such loads too severely reduce the workspace of the robot. The objects used and their masses are shown in Fig. 5. Given one of these objects, the task was to place the object into the cup corresponding to its mass. For each trial, a human assists the manipulator with grabbing the object, then the manipulator performs KL-MPC with load estimation (Algorithm 1) while following a circular reference trajectory for 15 seconds. After 15 seconds, load estimation stops, and a “drop-off” reference trajectory is selected that will move the end effector towards the cup corresponding to the most recent payload estimate. The manipulator then uses KL-MPC to follow the “drop-off” trajectory and deposits the object into the cup. This cycle repeats until all 5 objects are sorted into the proper cup. Using this strategy, the manipulator properly sorted 5 out of 5 objects in 2 separate trials, using a different set of objects each time (see Fig. 5). Footage of these trials can be seen in the supplementary video file.

V. DISCUSSION AND CONCLUSION

This letter uses a Koopman operator based approach to model and estimate a variable payload of a soft continuum manipulator arm and employs this knowledge to improve control performance. Our work confirms that incorporating knowledge about the payload into the model improves tracking accuracy and makes the controller more robust to changes in the loading conditions. In Experiment 1, the KL-MPC controller, which incorporated the payload value, reduced the RMSE tracking error averaged over all payloads by approximately 33% and the standard deviation by about 77% compared to the K-MPC controller that did not utilize information about the payload (see Table I). The tracking performance of the KL-MPC controller was more consistent across all loads, whereas the K-MPC controller achieved its best performance near of the middle of the load range. This is likely a result of the least-squares identification process which yields a model that minimizes the average prediction error over all snapshot data irrespective of the different loading conditions.

By combining estimation, modeling, and control into a single MPC algorithm (Algorithm 1), we demonstrated the effectiveness of our approach to improve control accuracy under unknown loading conditions. We first tracked periodic trajectories with an unknown payload. Since the controller needs some time

to establish an accurate estimate of the load, the tracking error gradually decreases over time as the load estimate becomes more reliable. After approximately 20 seconds, the tracking error decreased to less than 30 mm, which was about equal to the error with a known load value. It is notable that this approach was capable of estimating loads other than those presented in the training data set that was used during model-identification. This suggests that, despite the fact that the approach is data-driven, the identified Koopman model is able to capture the actual physical effect of various loading conditions.

As a final demonstration, we implemented successful pick-and-(mass-based)-place object manipulation using the same algorithm. Unknown objects were successfully sorted by mass, taking advantage of the fact that the payload estimate was accurate enough to choose the correct container for each object and that the tracking error of the “drop-off” trajectory was small enough not to miss the cup. This required a payload estimate accuracy of less than 50 grams, and a tracking error accuracy of less than 45 mm (the radius of the cups).

While the manipulator exhibited sufficient accuracy to complete this task, several modifications could be made to the robot and controller to improve performance even further. First, the workspace of the manipulator could be greatly enlarged by replacing some of the current actuators with more powerful ones just by increasing the diameter of the PAMs [28]. Second, a model and controller could be identified with a shorter sampling time, which would enable the model to account for higher frequency behavior and track more dynamic trajectories. This could be achieved by making upgrades to our computational hardware and optimizing our code. Even with these changes, a fundamental limitation of any purely data-driven approach is an inability to generalize beyond observed data. Supplementing the Koopman model with a physics-based model prior could make the controller more robust to disturbances not present in the data.

So far, our approach has only been validated on one specific soft robot, model predictive controller, and type of loading/disturbance. Future work should investigate extensions of the approach applied to other systems, controllers, and disturbances such as continuously varying payload and incidental contact. Beyond specifying the inputs and outputs, no system knowledge was necessary in the implementation, and the tuning of algorithmic parameters such as the type and number of basis functions was minimal. We thus believe that the approach will readily extend to other systems and lays a foundation towards enabling the widespread use of automated soft manipulators in real-world applications.

REFERENCES

- [1] I. Abraham and T. D. Murphey, “Active learning of dynamics for data-driven control using koopman operators,” *IEEE Trans. Robot.*, vol. 35, no. 5, pp. 1071–1083, Oct. 2019.
- [2] I. Abraham, G. de laTorre, and T. Murphey, “Model-based control using koopman operators,” in *Proc. Robot.: Sci. Syst.*, Cambridge, Massachusetts, USA, Jul. 2017. doi: [10.15607/RSS.2017.XIII.052](https://doi.org/10.15607/RSS.2017.XIII.052).
- [3] J. R. Amend, E. Brown, N. Rodenberg, H. M. Jaeger, and H. Lipson, “A positive pressure universal gripper based on the jamming of granular material,” *IEEE Trans. Robot.*, vol. 28, no. 2, pp. 341–350, Apr. 2012.
- [4] S. Boyd and L. Vandenberghe, *Convex Optimization*. New York, NY, Cambridge Univ. Press, 2004.

- [5] D. Bruder, "Towards a Universal Modeling and Control Framework for Soft Robots," *PhD thesis*, 2020.
- [6] D. Bruder, B. Gillespie, C. D. Remy, and R. Vasudevan, "Modeling and control of soft robots using the koopman operator and model predictive control," in *Proc. Robotics: Sci. Syst.*, Freiburg/Breisgau, Germany, 2019a, doi: [10.15607/RSS.2019.XV.060](https://doi.org/10.15607/RSS.2019.XV.060).
- [7] D. Bruder, C. D. Remy, and R. Vasudevan, "Nonlinear system identification of soft robot dynamics using koopman operator theory," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 6244–6250.
- [8] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. New York, NY, Cambridge Univ. Press, 2019.
- [9] M. Budišić, R. Mohr, and I. Mezić, "Applied koopmanism," *Chaos: An Interdiscipl. J. Nonlinear Sci.*, vol. 22, no. 4, 2012, Art no. 047510.
- [10] M. Cianchetti, T. Ranzani, G. Gerboni, I. De Falco, C. Laschi, and A. Menciassi, "Stiff-flop surgical manipulator: Mechanical design and experimental characterization of the single module," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2013, pp. 3576–3581.
- [11] A. Colomé, D. Pardo, G. Alenya, and C. Torras, "External force estimation during compliant robot manipulation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2013, pp. 3535–3540.
- [12] N. Correll *et al.*, "Analysis and observations from the first amazon picking challenge," *IEEE Trans. Automat. Sci. Eng.*, vol. 15, no. 1, pp. 172–188, Jan. 2018.
- [13] T.G. Thuruthel, Y. Ansari, E. Falotico, and C. Laschi, "Control strategies for soft robotic manipulators: A survey," *Soft Robot.*, vol. 5, no. 2, pp. 149–163, 2018.
- [14] LLC Gurobi Optimization, "Gurobi optimizer reference manual," 2021. [Online]. Available: URL <http://www.gurobi.com>
- [15] M. W. Hannan and I. D. Walker, "The elephant trunk manipulator, design and implementation," in *Proc. IEEE/ASME Int. Conf. Adv. Intell. Mechatronics Proc. (Cat. No 01TH8556)*, vol. 1, 2001, pp. 14–19.
- [16] J. Hughes, U. Culha, F. Giardina, F. Guenther, A. Rosendo, and F. Iida, "Soft manipulators and grippers: A review," *Front. Robot. AI*, vol. 3, Art. no. 69, 2016.
- [17] P. Hyatt, D. Wingate, and M. D. Killpack, "Model-based control of soft actuators using learned non-linear discrete-time models," *Front. Robot. AI*, vol. 6, Art. no. 22, 2019, doi: [10.3389/frobt](https://doi.org/10.3389/frobt).
- [18] M. Korda and I. Mezić, "Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control," *Automatica*, vol. 93, pp. 149–160, 2018.
- [19] G. Mamakoukas, M. Castano, X. Tan, and T. Murphey, "Local koopman operators for data-driven control of robotic systems," in *Proc. Robot.: Sci. Syst.*, Freiburg/Breisgau, Germany, 2019, doi: [10.15607/RSS.2019.XV.054](https://doi.org/10.15607/RSS.2019.XV.054).
- [20] MATLAB, version 7.10.0 (R2017a). *The MathWorks Inc.*, Natick, Massachusetts, 2017.
- [21] A. Mauroy and J. Goncalves, "Koopman-based lifting techniques for nonlinear systems identification," *IEEE Trans. Autom. Control*, vol. 65, no. 6, pp. 2550–2565, Jun. 2020.
- [22] W. McMahan, B. A. Jones, and I. D. Walker, "Design and implementation of a multi-section continuum robot: Air-octor," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2005, pp. 2578–2585.
- [23] D. P. Miller, "Assistive robotics: An overview," in *Proc. Assistive Technol. Artif. Intell.* Berlin, Heidelberg, Springer, 1998, pp. 126–136.
- [24] B. T. Phillips *et al.*, "A dexterous, glove-based teleoperable low-power soft robotic arm for delicate deep-sea biological exploration," *Sci. Rep.*, vol. 8, no. 1, 2018, Art no. 14779.
- [25] J. B. Rawlings and D. Q. Mayne, *Model Predictive Control: Theory and Design*. Madison, Wisconsin: Nob Hill Pub., 2009.
- [26] S. Satheeshbabu, N. K. Uppalapati, G. Chowdhary, and G. Krishnan, "Open loop position control of soft continuum arm using deep reinforcement learning," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 5133–5139.
- [27] T.G. Thuruthel, E. Falotico, F. Renda, and C. Laschi, "Model-based reinforcement learning for closed-loop dynamic control of soft robotic manipulators," *IEEE Trans. Robot.*, vol. 35, no. 1, pp. 124–134, Feb. 2019.
- [28] B. Tondu, "Modelling of the mckibben artificial muscle: A review," *J. Intell. Mater. Syst. Structures*, vol. 23, no. 3, pp. 225–253, 2012.
- [29] P. Van Overschee and B. L. De Moor, *Subspace Identification for Linear Systems: Theory-Implementation-Applications*. Berlin, Heidelberg, Springer Sci. Bus. Media, 2012.
- [30] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data-driven approximation of the koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [31] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. A. Funkouser, "Tossingbot: Learning to throw arbitrary objects with residual physics," in *Proc. Robot.: Sci. Syst.*, Freiburg/Breisgau, Germany, Jun. 2019, doi: [10.15607/RSS.2019.XV.004](https://doi.org/10.15607/RSS.2019.XV.004).